## Q3 – Engineer the hashCode–equals Contract (6 Marks, K4)

> All numbers below were measured by me in a Java sandbox (2-core cloud machine, averaged over repeated runs after a warm-up). Absolute times will differ on your computer, so **re-run every program and report your own figures**; the trends and ratios are what matter. Predictions were derived from theory, but the assignment requires *you* to write your own prediction **before** you run each experiment, so write yours first. Each screenshot must be linked to the test it shows.

### 1. Complete the Code – before and after TODO 1

**Before TODO 1** (class has `equals` only, as given): output

```
Unique students: 3
```

**After TODO 1** (`hashCode()` added): output

```
Unique students: 2
```

matches the expected output. The completed class (`Attendance.java`):

```
import java.util.HashSet;
import java.util.Set;

class Student {
    private final String regNo;
    private final String name;

    Student(String regNo, String name) { this.regNo = regNo; this.name
= name; }

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        return regNo.equals(((Student) o).regNo);
    }

    // TODO 1 (completed): consistent with equals(), uses the same
field (regNo)
    @Override public int hashCode() { return regNo.hashCode(); }

    @Override public String toString() { return regNo + " " + name; }
}

public class Attendance {
    public static void main(String[] args) {
        Set<Student> present = new HashSet<>();
        present.add(new Student("192311001", "Arun"));    // scanned
at Gate 1
```

```
        present.add(new Student("192311002", "Divya"));
        present.add(new Student("192311001", "Arun"));    // scanned
again at Gate 3
        System.out.println("Unique students: " + present.size());
    }
}
```

**Why the difference:** a `HashSet` first calls `hashCode()` to choose a bucket and calls `equals()` only on objects *in the same bucket*. Without an override, `hashCode()` is the identity hash, different for each `new Student(...)` object, so the two scans of `192311001` almost always go to different buckets and `equals()` is never even asked whether they are the same student: the set keeps both and reports 3. (If two identity hashes happened to fall in the same bucket the duplicate would be caught, so the broken program can give the right answer by luck, which is what makes the bug dangerous.)

## 2. Designed tests, with predictions made before running

`ContractTests.java` runs the tests (the prediction text is printed before each result):

```
import java.util.*;

/** Three designed tests, with the prediction printed BEFORE the
result. */
public class ContractTests {
    static class Bad {      // equals without hashCode
        final String regNo; Bad(String r) { regNo = r; }
        @Override public boolean equals(Object o) { return o
instanceof Bad && regNo.equals(((Bad) o).regNo); }
    }
    static class Good {     // equals + hashCode
        final String regNo; Good(String r) { regNo = r; }
        @Override public boolean equals(Object o) { return o
instanceof Good && regNo.equals(((Good) o).regNo); }
        @Override public int hashCode() { return regNo.hashCode(); }
    }
    static class Mutable {  // regNo is NOT final, used in hashCode
        String regNo; Mutable(String r) { regNo = r; }
        @Override public boolean equals(Object o) { return o
instanceof Mutable && regNo.equals(((Mutable) o).regNo); }
        @Override public int hashCode() { return regNo.hashCode(); }
    }
    static class Student { String regNo, name; Student(String r,
String n) { regNo = r; name = n; } public String toString() { return
regNo + " " + name; } }

    public static void main(String[] a) {
        // Test 1: distinct students only (no repeats) -> works even
without hashCode
        Set<Bad> t1 = new HashSet<>(); for (int i = 1; i <= 5; i++)
t1.add(new Bad("R" + i));
```

```
        System.out.println("Test 1  distinct scans only, equals w/o
hashCode. Prediction: 5.   Observed: " + t1.size());

        // Test 2: repeated scans of the same students
        Set<Bad> t2 = new HashSet<>(); Set<Good> t2g = new
HashSet<>();
        for (int round = 0; round < 3; round++) for (int i = 1; i <=
5; i++) { t2.add(new Bad("R" + i)); t2g.add(new Good("R" + i)); }
        System.out.println("Test 2  5 students scanned 3 times each.
Prediction: without hashCode ~15 (duplicates), with hashCode 5.
Observed: without=" + t2.size() + "  with=" + t2g.size());
        Set<Bad> t2b = new HashSet<>(); Bad one = new Bad("X1");
t2b.add(one);
        System.out.println("        contains(new Bad(\"X1\")) equals-
wise true but: " + t2b.contains(new Bad("X1")) + "   contains(same
object): " + t2b.contains(one));

        // Test 3 (own): change the field used in hashCode after
insertion
        Set<Mutable> t3 = new HashSet<>(); Mutable m = new
Mutable("A1"); t3.add(m); m.regNo = "B2";
        System.out.println("Test 3  mutate regNo after add.
Prediction: size 1, but contains(m) false and contains(new
Mutable(\"B2\")) false (object is in the wrong bucket).");
        System.out.println("        Observed: size=" + t3.size() + "
contains(m)=" + t3.contains(m) + "  contains(B2)=" + t3.contains(new
Mutable("B2")) + "  contains(A1)=" + t3.contains(new Mutable("A1")));
        t3.add(new Mutable("B2")); System.out.println("        adding
another B2 -> size=" + t3.size() + " (duplicate of the mutated element
is accepted)");

        // Test 4 (own): TreeSet whose compareTo is inconsistent with
equals
        TreeSet<Student> byName = new TreeSet<>(Comparator.comparing(s
-> s.name));    // compares only name
        byName.add(new Student("R1", "Arun")); byName.add(new
Student("R2", "Arun"));
        System.out.println("Test 4  TreeSet comparing only name, two
different regNo, same name. Prediction: size 1.  Observed: " +
byName.size() + "  " + byName);
    }
}
```

| # | Test | Prediction and reason | Observed |
|---|---|---|---|
| 1 | **Works correctly:** 5 distinct students, each scanned | Size **5**. With no repeated scans there is no duplicate to | **5** |

| # | Test | Prediction and reason | Observed |
|---|---|---|---|
| | once, class with `equals` but no `hashCode` | detect, so the missing `hashCode` is not visible. | |
| 2 | **Exposes duplicates:** the same 5 students scanned 3 times each | Without `hashCode`: about **15** (each scan is a new identity hash, so each lands in an unrelated bucket). With `hashCode`: **5**. | without = **15**, with = **5** |
| 2b | `contains(new Bad("X1"))` after adding an *equal* object | `false`, although `equals` says they are equal, because a different bucket is searched. | **false** (and `true` for the very same object) |
| 3 | **My own:** change `regNo` (used in `hashCode`) *after* adding the object to the set | Size stays 1, but the object is stored in the bucket for the **old** hash, so `contains(m)` should fail (it now looks in the new bucket). | size = 1, `contains(m)` = **false**, `contains(new B2)` = **false**, `contains(new A1)` = **false** (hash matches, but `equals` compares with the changed `regNo`, so it fails); adding another `B2` gave **size 2** |
| 4 | **My own:** `TreeSet` whose comparator compares only `name` (inconsistent with `equals`), two different students named | Size **1**, because `TreeSet` uses `compareTo`/the comparator (not `equals`), and 0 means "same element". | **1** (`[R1 Arun]`; the second student was silently dropped) |

| # | Test | Prediction and reason | Observed |
|---|---|---|---|
| | Arun | | |

## 3. Inquiry – explanation

**Why duplicates appear:** `equals` was overridden but `hashCode` was not, which breaks the contract *"if `a.equals(b)` then `a.hashCode() == b.hashCode()`"*. `HashSet` trusts the hash to pick a bucket, so equal objects with different hashes live in different buckets and are never compared.

**How `HashSet` uses `hashCode()` and `equals()`:** `add(x)` computes `h = x.hashCode()`, maps it to a bucket (index = a function of `h` and the table size), then compares `x` with every element already in that bucket, first by cached hash and reference, then with `x.equals(y)`. If one is equal, `x` is a duplicate and is not added. `contains` and `remove` follow the same path. A good hash spreads objects over many buckets so that the expected number of `equals` calls is O(1).

**What happens when a field used in `hashCode()` changes after insertion:** the element stays in the bucket for the old hash, but lookups compute the new hash and search another bucket (test 3): the element is **lost** (`contains` is false, `remove` fails) yet still counted in `size()` and still visible when iterating, and a "new" equal object can be added again (size 2). Rule: use **immutable** fields (`final`) in `hashCode`/`equals`, or never mutate an object while it is in a hash-based collection.

**How to correct the class and verify the correction:**

```
@Override public boolean equals(Object o) { if (this == o) return
true; if (!(o instanceof Student)) return false; return
regNo.equals(((Student) o).regNo); }
@Override public int hashCode() { return regNo.hashCode(); }     //
same field(s) as equals; or Objects.hash(regNo)
```

Verification: (1) re-run the program: 2 unique students; (2) test 2: size 5 for 15 scans; (3) assert `a.equals(b)` ⇒ `a.hashCode()==b.hashCode()` for sample pairs; (4) check `contains(new Student(sameRegNo, otherName))` is `true`; (5) keep `regNo` `final`.

## 4. Alternative concept and complexity analysis

**`TreeSet<Student>` with a comparator on `regNo`, or `HashMap<String, Student>` keyed by `regNo`:**

```
TreeSet<Student> set = new TreeSet<>(Comparator.comparing(s ->
s.regNo));   // needs only a consistent comparator; no hashCode
Map<String, Student> map = new HashMap<>();  map.putIfAbsent(s.regNo,
s);   // String already has a correct hashCode/equals
```

`SetCompare.java` measured n unique students, each scanned twice (2n scans), shuffled:

```java
import java.util.*;

/** Alternative: TreeSet with a comparator on regNo,
HashMap<String,Student> keyed by regNo. */
public class SetCompare {
    record S(String regNo, String name) {}
    static double ms(Runnable r) { long t = System.nanoTime();
r.run(); return (System.nanoTime() - t) / 1e6; }
    public static void main(String[] a) {
        System.out.printf("%9s | %10s %10s %10s   (ms for n scans,
each student scanned twice) | memory KB (retained)%n", "n unique",
"HashSet", "TreeSet", "HashMap");
        for (int n : new int[]{10_000, 100_000, 1_000_000}) {
            S[] scans = new S[2 * n]; for (int i = 0; i < n; i++)
{ scans[i] = new S("R" + i, "N" + i); scans[n + i] = new S("R" + i,
"N" + i); }
            Collections.shuffle(Arrays.asList(scans), new Random(1));
            for (int w = 0; w < 2; w++) { Set<S> h = new HashSet<>();
for (S s : scans) h.add(s); }
            double hs = 0, ts = 0, hm = 0; int reps = 3;
            long memH = 0, memT = 0, memM = 0;
            for (int r = 0; r < reps; r++) {
                Set<S> h = new HashSet<>(); hs += ms(() -> { for (S s
: scans) h.add(s); });
                TreeSet<S> t = new
TreeSet<>(Comparator.comparing(S::regNo)); ts += ms(() -> { for (S s :
scans) t.add(s); });
                Map<String, S> m = new HashMap<>(); hm += ms(() -> {
for (S s : scans) m.putIfAbsent(s.regNo(), s); });
                if (h.size() != n || t.size() != n || m.size() != n)
throw new IllegalStateException();
            }
            Runtime rt = Runtime.getRuntime();
            System.gc(); long b = rt.totalMemory() - rt.freeMemory();
Set<S> h = new HashSet<>(); for (S s : scans) h.add(s); System.gc();
memH = rt.totalMemory() - rt.freeMemory() - b;
            System.gc(); b = rt.totalMemory() - rt.freeMemory();
TreeSet<S> t = new TreeSet<>(Comparator.comparing(S::regNo)); for (S s
: scans) t.add(s); System.gc(); memT = rt.totalMemory() -
rt.freeMemory() - b;
            System.out.printf("%9d | %10.1f %10.1f %10.1f   | HashSet~
%d  TreeSet~%d%n", n, hs / reps, ts / reps, hm / reps, memH / 1024,
memT / 1024);
        }
    }
}
```

| n unique | `HashSet<Student>` | `TreeSet` (comparator on regNo) | `HashMap<String,Student>` |
|---|---|---|---|
| 10,000 | 6.1 ms | 8.0 ms | 6.0 ms |
| 100,000 | 72.9 ms | 276.4 ms | 39.9 ms |
| 1,000,000 | 1,883 ms | 5,544 ms | 601 ms |

(Time for the 2n scans in total. The program also tried to measure retained memory by forcing garbage collection, but those readings were unreliable (negative numbers), so I do not report them; the space analysis below is analytical.)

- **Time per scan:** `HashSet`/`HashMap`: **O(1) average** (O(n) worst case if every key collides; since Java 8 a heavily colliding bucket becomes a balanced tree, giving O(log n)). `TreeSet`: **O(log n)** always (a red-black tree), which is visible: at n = 1,000,000 the tree was about 3–9× slower than the hash structures. `HashSet` time grew ×12 when n grew ×10 (6.1 → 72.9 ms) and ×26 when n grew another ×10 (→ 1,883 ms, with more garbage-collection pressure at this size); `TreeSet` grew ×35 then ×20, consistent with n log n plus cache misses. Single measurements at this scale are noisy, so repeat them.
- **`HashMap<String,Student>` was faster than `HashSet<Student>`** (601 vs 1,883 ms at 1 M). Probable reason (I did not isolate it): my `Student` was a record, whose `hashCode`/`equals` combine *both* fields, while the map uses the cached `String.hashCode` of the key; memory pressure from many objects also adds garbage-collection time.
- **Space for n unique students:** all three are **O(n)**. `HashSet` is backed by a `HashMap`, so it stores a table of about 1.3–2 n slots plus one 32-byte entry object per element; `HashMap` is the same plus the key (the `regNo` String is shared with the Student if you reuse it); `TreeSet` stores one tree entry per element (about 40 bytes: key, left, right, parent, colour), with no spare table. The elements themselves are the same.
- **Choosing:** keep `HashSet` (with a correct `hashCode`) when only uniqueness matters; use `HashMap<String,Student>` when you also look students up by `regNo`; use `TreeSet` only if you need sorted order or range queries, and then the comparator must be **consistent with equals** (test 4 shows what goes wrong otherwise).